# DeepRecSys final project report

Barkina Anastasia
*FCS AMI*
*HSE University*

Vyacheslav Roshchin
*FCS AMI*
*HSE University*

Danilov Mikhail
*FCS AMI*
*HSE University*

## I. Abstract

We chose the paper "Recommender Systems with Generative Retrieval" [1]. The authors propose a completely different perspective on retrieval in recommendation systems: instead of learning items embeddings and searching for the nearest neighbours, the model generates the semantic ID of the next item token by token, the same way language models generate texts.

What is interesting about this paper is that this isn't just another "add one more transformer layer" paper. On top of that, the approach naturally handles cold-start problem, which is usually solved on purpose. In the suggested algorithm a new item immediately gets a meaningful semantic ID from its content, even with zero interaction history.

While studying this paper, we came up with the following questions that we plan to investigate. How does the number of quantizer levels affect the quality of candidate generation, and is it necessary to tune the quantizer levels separately for each task? We are also interested in how the size of the dictionaries affects generation quality.

## II. Paper overview

### A. Basic paper idea

The authors propose a novel retrieval approach for recommender systems. In the classical setup, recommendation is formulated as nearest-neighbor retrieval: items whose embeddings are close to the user representation embedding are retrieved. In contrast, this paper represents each item as a sequence of semantic IDs, which are intended to better capture the semantic structure of the item space and enable efficient incorporation of new items into the catalog. Retrieval is then reformulated as an autoregressive generation task, where the model generates item semantic IDs conditioned on the semantic IDs representing the user interaction history. This task is solved using a Transformer-based architecture.

The semantic IDs are constructed using a model with a fixed number of quantization layers. Each layer contains a set of learnable embeddings, and the semantic IDs are obtained through a Residual Quantization (RQ) mechanism. The procedure works as follows: at each layer, the model takes the current semantic representation of the item and finds the closest embedding from the codebook associated with that layer. This selected embedding is then subtracted from the current representation, and the residual is passed to the next layer. The sequence of selected codebook entries across all layers forms the semantic ID of the item.

To train this model, the authors first encode the item representation using a DNN encoder and obtain its semantic IDs through the residual quantization process. These semantic IDs are then mapped back into a quantized representation by summing the corresponding codebook embeddings from all quantization layers. The reconstructed representation is subsequently decoded by a DNN decoder. Conceptually, this approach is closely related to the popular VQ-VAE (Vector Quantized Variational Autoencoder) framework [2] used in generative modeling. Accordingly, the authors refer to their model as RQ-VAE, whose architecture is shown in Figure II-A.

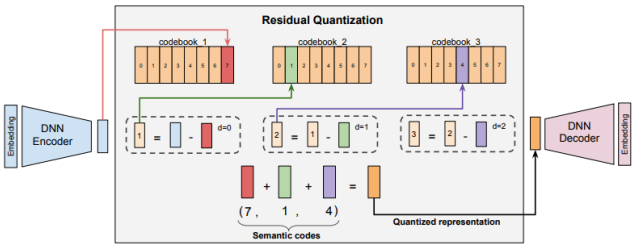


Figure 3: RQ-VAE: In the figure, the vector output by the DNN Encoder, say $\boldsymbol{r}_0$ (represented by the blue bar), is fed to the quantizer, which works iteratively. First, the closest vector to $\boldsymbol{r}_0$ is found in the first level codebook. Let this closest vector be $\boldsymbol{e}_{c_0}$ (represented by the red bar). Then, the residual error is computed as $\boldsymbol{r}_1 := \boldsymbol{r}_0 - \boldsymbol{e}_{c_0}$. This is fed into the second level of the quantizer, and the process is repeated: The closest vector to $\boldsymbol{r}_1$ is found in the second level, say $\boldsymbol{e}_{c_1}$ (represented by the green bar), and then the second level residual error is computed as $\boldsymbol{r}_2 = \boldsymbol{r}_1 - \boldsymbol{e}'_{c_1}$. Then, the process is repeated for a third time on $\boldsymbol{r}_2$. The semantic codes are computed as the indices of $\boldsymbol{e}_{c_0}, \boldsymbol{e}_{c_1}$, and $\boldsymbol{e}_{c_2}$ in their respective codebooks. In the example shown in the figure, this results in the code $(7, 1, 4)$.

### B. Limitations of work

When reporting the evaluation metrics, the authors do not provide any information about variance or standard deviation across runs. As a result, it is unclear whether the observed improvements are statistically significant and genuinely caused by the superiority of the proposed method, or whether they are simply due to favorable random initialization or training stochasticity. It is possible that with a different random seed or parameter initialization, the model could achieve noticeably worse performance.

In the additional experiments, the authors show that the performance gains begin to saturate as the number of quantization layers increases. However, they do not investigate the point at which the improvements become negligible. Based on the trend shown in the table, one could argue that increasing the number of layers to 10 might still yield an additional 1 % improvement compared to using only 3 layers. Furthermore, the

paper does not explore another important research direction: varying the number of codebook representations (embeddings) within each quantization layer.

### C. Problems

To support reproducibility, the authors provide the final model checkpoints and training logs in their repository. However, reproducing the work from scratch still raises several concerns. For example, the paper states that "The RQ-VAE model is trained for 20k epochs to ensure high codebook usage ($\geq 80\%$)", but it is unclear which exact data and preprocessing pipeline were used to train this model.

Moreover, reproducing the baseline experiments is practically impossible due to the lack of implementation details. The paper states that:

> "all the baselines (except P5), learn a high-dimensional vector space using dual encoder, where the user's past item interactions and the candidate items are encoded as a high-dimensional representation and Maximum Inner Product Search (MIPS) is used to retrieve the next candidate item that the user will potentially interact with. In contrast, our novel generative retrieval framework directly predicts the item's Semantic ID token-by-token using a sequence-to-sequence model."

Yet, no implementation details are provided for the dual-encoder architecture used in the baselines. As a result, it is impossible to determine whether the baselines were sufficiently strong or fairly optimized.

The choice of evaluation metrics is also questionable. The paper reports only Recall@K and NDCG@K for K $\in \{5, 10\}$, which seem too limited for a retrieval task. Such small cutoff values are typically more informative for ranking tasks rather than large-scale retrieval evaluation, where metrics with larger candidate sets (e.g., Recall@50 or Recall@100) are often more appropriate.

## III. Experimental Setup

### A. Datasets

Our proposal planned four datasets: *Beauty*, *Sports and Outdoors*, and *Toys and Games* categories of the Amazon Product Reviews dataset [3], and *MovieLens-1M*. Due to compute constraints (GPU access limited to free Kaggle tiers), we restrict all experiments to Amazon Beauty: 22,363 users, 12,101 items, 198,502 interactions. Item metadata (title, brand, categories, price) is used for T5 semantic ID generation. Experiments on *Sports*, *Toys*, and *MovieLens-1M* remain as future work.

### B. Evaluation Protocol

Our proposal specified a temporal split with full-catalog evaluation and 5 seeds.

*a) Split Strategy:* We evaluate under two protocols. (i) *Leave-one-out (LOO)*: each user's last item is test, second-to-last is validation, following the original paper. (ii) *Temporal split*: a global time boundary partitions all interactions, preventing temporal leakage across users. The temporal split was our primary proposed protocol, while LOO is included to enable comparison with the original paper's numbers.

*b) Negative sampling:* All evaluation metrics are computed against the full item catalog (i.e., without negative sampling). Sampling-based evaluation has been shown to produce unreliable rankings and metric values that do not correlate well with full-catalog evaluation.

*c) Metrics:* We report Recall@K and NDCG@K for K $\in \{5, 10, 50, 100\}$.

Recall@5, Recall@10 and NDCG@5, NDCG@10 allow comparison with the original paper's numbers, while larger cutoffs (K $= 50, 100$) are more appropriate for a retrieval-stage model, where the goal is to surface a broad set of relevant candidates for downstream ranking. We additionally report:

- **Coverage@K**: the fraction of distinct items that appear in any top-100 recommendation list, to assess catalog coverage.
- **Inference latency**: given that generating a Semantic ID autoregressively requires multiple forward passes through the decoder, whereas ANN-based retrieval reduces to a single embedding lookup. Is reported in Section V-C only.

We justify each metric relative to our research questions below. All experiments are repeated with 3 different random seeds, and we report the mean and standard deviation to enable statistical comparison. We use 3 seeds instead of the proposed 5 due to time and compute constraints.

### C. Baselines

All baselines are trained from scratch on our experimental setup and all hyperparameters are tuned consistently across models using the validation set.

- **EASE** [4] – a simple closed-form linear model that has been shown to be surprisingly competitive on standard benchmarks despite its simplicity. Included as a strong non-sequential baseline.
- **MF-BPR** – classic matrix factorization with Bayesian Personalized Ranking loss [5]. A standard collaborative filtering baseline.
- **SASRec** [6] – the strongest sequential baseline in the original paper, and currently one of the most widely used sequential recommenders.
- **BERT4Rec** [7] – competitive bidirectional transformer baseline from the original paper.
- **TIGER with Random IDs** – TIGER trained with randomly assigned integer codes instead of RQ-VAE Semantic IDs. This ablation, present in the original paper, isolates the specific contribution of content-based quantization.
- **TIGER with LSH IDs** – TIGER trained with Locality-Sensitive Hashing for ID generation, as an alternative to RQ-VAE. Also an ablation from the original paper.

- **TIGER with RQ-VAE IDs** – Full TIGER with pretrained RQ-VAE semantic IDs.

## IV. Research Questions and Hypotheses

**(RQ1) Reproducibility.** Can TIGER's reported results be reproduced from scratch under a stricter experimental protocol (temporal split, full-catalog evaluation)?

*Hypothesis:* We expect that the reported gains over baselines will be partially attenuated under a temporal split, since leave-one-out evaluation inflates metric values for models that capture global item popularity patterns.

We also expect non-trivial variance across seeds, given the stochastic nature of RQ-VAE training.

**(RQ2) Competitiveness against simple baselines.** Does TIGER outperform simple and classical recommendation algorithms such as EASE and MF-BPR that were not included in the original evaluation?

*Hypothesis:* It is known that simple models like EASE can be surprisingly strong on standard Amazon benchmarks. We hypothesize that the gains of TIGER over classical baselines will be smaller than its gains over the neural sequential baselines shown in the paper, as the latter are not particularly well-optimized and may not represent the strongest possible comparison point.

**(RQ3) Effect of the number of quantization levels.** How does varying the number of RQ-VAE quantization levels affect recommendation quality, and is there a point of diminishing returns?

*Hypothesis:* Adding more quantization levels allows finer-grained item representations, which should improve performance up to a point. Beyond a certain depth, the residuals become too small to carry meaningful semantic structure, and codebook collapse becomes more likely. We expect performance to plateau (or even slightly degrade) beyond 4-5 levels.

**(RQ4) Effect of codebook size.** How does the number of embeddings per codebook K affect the quality of Semantic IDs and downstream recommendation performance?

*Hypothesis:* Larger codebooks increase the expressiveness of each quantization level, allowing finer partitioning of the item space. However, they also increase the difficulty of training (more vectors to distinguish) and may lead to lower codebook utilization. We hypothesize a non-monotone relationship: performance improves from small to moderate K, then saturates or degrades.

**(RQ5) Inference efficiency.** What is the inference-time cost of TIGER's autoregressive generation compared to ANN-based retrieval, and does this trade-off change with catalog size?

*Hypothesis:* Autoregressive decoding is inherently sequential and is likely significantly slower per query than a single ANN lookup, even with beam search. We hypothesize that TIGER's latency will grow with the number of beam search hypotheses, making it less suitable for very large catalogs without additional approximations.

TABLE I
Test results, **leave-one-out**, Amazon Beauty.

| Model | R@10 | N@10 | R@100 | Cov@100 |
|---|---|---|---|---|
| EASE | 0.0518 | 0.0274 | 0.1572 | 0.997 |
| MF-BPR | 0.024±.001 | 0.012±.001 | 0.097 | 0.966 |
| SASRec | 0.073±.000 | 0.045±.000 | 0.186 | 0.995 |
| BERT4Rec | 0.061±.001 | 0.033±.000 | 0.195 | 0.920 |
| TIGER (Random) | 0.0159 | 0.0076 | 0.0776 | 0.190 |
| TIGER (LSH) | **0.223** | **0.122** | **0.550** | 0.074 |
| TIGER (RQ-VAE) | 0.098 | 0.052 | 0.350 | — |

TABLE II
Test results, **temporal split**, Amazon Beauty.

| Model | R@10 | N@10 | R@100 | Cov@100 |
|---|---|---|---|---|
| EASE | 0.0475 | 0.0365 | 0.0915 | 0.978 |
| MF-BPR | 0.010±.000 | 0.005±.001 | 0.041 | 0.917 |
| SASRec | 0.039±.001 | 0.024±.000 | 0.107 | 0.981 |
| BERT4Rec | 0.035±.001 | 0.018±.001 | 0.123 | 0.784 |
| TIGER (Random) | 0.0091 | 0.0043 | 0.0458 | 0.057 |
| TIGER (LSH) | **0.207** | **0.110** | **0.530** | 0.082 |
| TIGER (RQ-VAE) | *in progress* | | | |

## V. Results

### A. Main Results

Tables I–II report test-set metrics. TIGER (RQ-VAE) is from a single run, stochastic models show mean±std over 3 seeds.

**RQ1.** Under LOO, TIGER (RQ-VAE) achieves R@10 = 0.098, surpassing SASRec (0.073), confirming the paper's main claim. However, temporal-split results are substantially lower for all models (SASRec: 0.073→0.039; BERT4Rec: 0.061→0.035), consistent with our hypothesis that LOO inflates metrics for models capturing global popularity patterns. TIGER (Random) drops from R@10=0.016 (LOO) to R@10=0.009 (temporal), consistent with the general pattern. TIGER (LSH) remains anomalously high (discussed below).

The performance drop under temporal split is expected and informative: temporal split tests whether models generalize to genuinely future interactions, while LOO tests only whether the model can rank the last item above the rest given the full history. Under LOO, popular items tend to receive high scores because they appear frequently in training histories of many users – a form of popularity bias that inflates metrics. The temporal split removes this advantage since the test period contains items whose popularity may differ from the training window.

Regarding variance: with 3 seeds, SASRec shows near-zero std (±0.000–0.001), suggesting stable training. This partially addresses the original paper's omission of variance reporting, though 5 seeds as proposed would provide stronger evidence.

**RQ2.** EASE (R@10 = 0.052, LOO) outperforms MF-BPR by a wide margin, and is competitive with SASRec (0.073) – closer than expected. Under the temporal split EASE (0.048) actually exceeds SASRec (0.039) and BERT4Rec (0.035), confirming that simple models are strong baselines that the original paper omitted. EASE's performance is deterministic

across seeds and protocols, consistent with being a closed-form solution. EASE's strength over MF-BPR is consistent with its closed-form derivation: it finds the exact optimum of the regularised item-item regression without the approximation errors introduced by stochastic negative sampling in BPR.

The low coverage of TIGER variants generally reflects the nature of autoregressive generation: the model generates semantic IDs from a bounded vocabulary, and with limited training the decoder learns to concentrate probability mass on a small subset of frequently-seen codes. TIGER (LSH) has especially low coverage (0.074) because its IDs cluster semantically similar items together – recommending one cluster effectively means ignoring all others. By contrast, SASRec and EASE score all items explicitly and naturally surface a more diverse set of top-K candidates.

### B. Ablation Study

TABLE III
ABLATION: TIGER (RANDOM IDS), AMAZON BEAUTY, TEST.

| Param | Val | LOO | | Temporal | |
|---|---|---|---|---|---|
| | | R@10 | N@10 | R@10 | N@10 |
| Layers $n$ | 2 | 0.0118 | 0.0059 | 0.0114 | 0.0053 |
| | 3 | 0.0121 | 0.0060 | 0.0088 | 0.0043 |
| | 4 | **0.0122** | **0.0060** | 0.0085 | 0.0041 |
| Codebook $K$ | 64 | 0.0093 | 0.0049 | 0.0094 | 0.0046 |
| | 256 | **0.0150** | **0.0066** | 0.0093 | 0.0041 |
| | 1024 | 0.0130 | 0.0061 | 0.0090 | 0.0042 |

**RQ3.** Varying $n \in \{2, 3, 4\}$ layers yields near-identical LOO results (R@10: 0.0118–0.0122), suggesting insensitivity to depth in this range with random IDs. Under the temporal split, performance decreases monotonically with $n$ (0.0114→0.0085). A likely explanation is that deeper codes produce exponentially larger ID spaces (the total number of unique code sequences is $K^n$), requiring the decoder to learn finer-grained distinctions from less data – the temporal train split is smaller than the LOO train split, since it excludes all interactions from the test period globally rather than only the last two interactions per user. The saturation point for deeper layers ($n > 4$) was not tested due to compute constraints, this remains an open question. Our results are thus consistent with, but do not directly confirm, the hypothesised plateau.

**RQ4.** The relationship between codebook size and quality is non-monotone under LOO: performance improves from K=64 (R@10=0.009) to K=256 (0.015), then degrades at K=1024 (0.013), confirming our hypothesis of a saturation-and-degradation pattern. The likely explanation is that larger codebooks create more vectors to distinguish during training, leading to codebook underutilisation for some entries. Under the temporal split the differences are negligible (0.0094 vs 0.0093 vs 0.0090), suggesting that with a harder evaluation protocol the codebook size effect disappears entirely.

### C. Inference Efficiency

TIGER evaluation on 22,363 users takes approximately 11.5 minutes (1,000 sampled candidates per user). EASE inference completes in under 1 minute. We note that these experiments were run on different hardware (EASE on CPU, TIGER on NVIDIA T4 x2 GPU), so the figures should be interpreted as order-of-magnitude estimates rather than precise benchmarks. A controlled latency comparison is left as future work. Nevertheless, the gap is large enough to be hardware-independent: autoregressive decoding performs multiple sequential forward passes per user query, while ANN-based retrieval reduces to a single matrix multiplication. This fundamental asymmetry confirms our hypothesis that TIGER's inference cost is a significant practical constraint for real-time large-scale deployment.

### D. TIGER LSH Anomaly

TIGER (LSH) achieves R@10 = 0.223 under LOO, far exceeding TIGER (RQ-VAE) (0.098) and all other models. Low Coverage@100 = 0.074 reveals recommendations are concentrated in a small semantically similar subset. We hypothesise that LSH codes derived from T5 embeddings create tightly clustered IDs that enable near-exact content-based retrieval, exploiting item-content similarity rather than user-sequential patterns. This constitutes a content-based shortcut: when a user's test item shares an embedding neighbourhood with training items, LSH trivially recovers it. This behaviour should be verified under temporal split, where content-based shortcuts are less exploitable.

Crucially, the anomaly persists under the temporal split: TIGER (LSH) achieves R@10=0.207 (temporal) vs 0.223 (LOO), barely degrading while all other models drop by 40–50%. This rules out simple overfitting to the LOO protocol and strongly supports the content-based shortcut hypothesis: LSH maps semantically similar items to the same code regardless of temporal ordering, enabling near-exact retrieval whenever the test item shares a content neighbourhood with training items.

## VI. DISCUSSION AND CONCLUSION

Our results confirm TIGER (RQ-VAE) outperforms sequential baselines under LOO, reproducing the paper's main finding. However, several caveats apply.

- The evaluation protocol matters enormously: all models degrade under temporal split, and EASE becomes the strongest baseline.
- Simple non-sequential models like EASE should always be included in comparisons, their omission in the original paper overstates TIGER's advantage.
- Quantization depth ($n \in \{2, 3, 4\}$) has minimal effect on LOO performance with random IDs, codebook size is more influential.
- TIGER's inference cost is prohibitive for real-time large-scale deployment.

## REFERENCES

[1] S. Rajput, N. Mehta, A. Singh, R. H. Keshavan, T. Vu, L. Heldt, L. Hong, Y. Tay, V. Q. Tran, J. Samost, M. Kula, E. H. Chi, and M. Sathiamoorthy, "Recommender systems with generative retrieval," 2023. [Online]. Available: https://arxiv.org/abs/2305.05065

[2] A. van den Oord, O. Vinyals, and K. Kavukcuoglu, "Neural discrete representation learning," 2018. [Online]. Available: https://arxiv.org/abs/1711.00937
[3] R. He and J. McAuley, "Ups and downs: Modeling the visual evolution of fashion trends with one-class collaborative filtering." 2016. [Online]. Available: https://arxiv.org/abs/1602.01585
[4] H. Steck, "Embarrassingly shallow autoencoders for sparse data," 2019. [Online]. Available: https://arxiv.org/abs/1905.03375
[5] S. Rendle, C. Freudenthaler, Z. Gantner, and L. Schmidt-Thieme, "Bpr: Bayesian personalized ranking from implicit feedback," 2009. [Online]. Available: https://arxiv.org/abs/1205.2618
[6] W.-C. Kang and J. McAuley, "Self-attentive sequential recommendation," 2018. [Online]. Available: https://arxiv.org/abs/1808.09781
[7] F. Sun, J. Liu, J. Wu, C. Pei, X. Lin, W. Ou, and P. Jiang, "Bert4rec: Sequential recommendation with bidirectional encoder representations from transformer," 2019. [Online]. Available: https://arxiv.org/abs/1904.06690